# SDLC – Corporate Training Lecture

Oct 14, 2026 · @Sujay

## Lecture overview

This session walks through the six phases of the Software Development Life Cycle (SDLC), using **Zerodha Kite** (a stock-trading app) as the running example, with Paytm recharge examples where they make a point clearer.

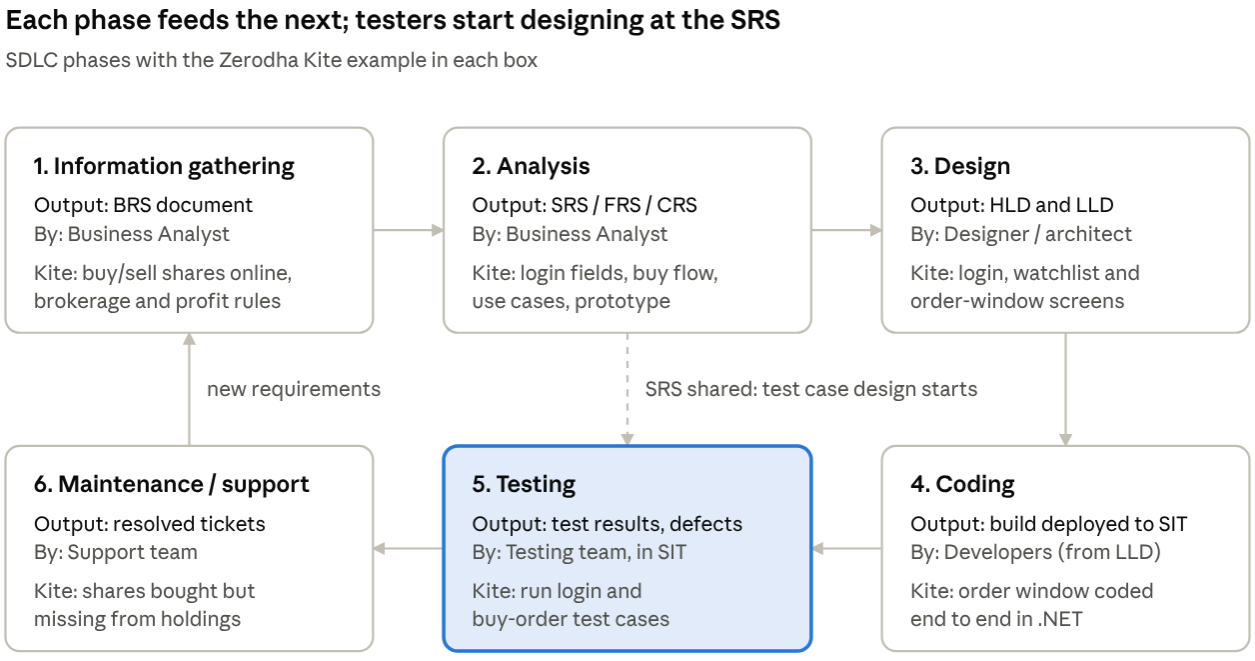
By the end of the session, participants should be able to:

- Name the six SDLC phases in order and the output (document or deliverable) of each.
- Explain the difference between the BRS and the SRS, and who prepares them.
- Describe what a tester does in each phase, and when testing actually starts.
- Answer the common interview questions on SDLC with confidence.

**Audience:** freshers and junior QA / testing professionals. **Duration:** about 60 minutes.

## What is SDLC?

**SDLC (Software Development Life Cycle)** is the step-by-step process a project team follows to build software, from collecting the client's requirement to supporting the live application. It has six phases, and the output of each phase is the input to the next.




SDLC phases · 6 stages with Zerodha Kite examples

Read it clockwise from phase 1. The dashed line is the key point for testers: test case design starts as soon as the SRS is shared, long before the build reaches SIT. The highlighted box is where testers spend most of their time.

## Phase 1 – Information gathering (BRS)

In this phase the Business Analyst (BA) collects requirements from the client and writes the **BRS – Business Requirement Specification**.

- The BA meets the client and collects the business requirements.
- The BA defines the BRS document, which holds the **total business requirement**.
- The platform is later developed according to this requirement.
- Output of the phase: the **BRS document**.

### Zerodha Kite example

The client (a stock broker) wants a platform where end users can buy and sell shares online. In the BRS, the BA captures the business need in business language, for example:

- Customers should be able to buy and sell shares from web and mobile.
- The broker earns money through a brokerage charge on each trade.
- The customer should see their profit or loss after charges.

**Business calculation captured in the BRS (illustrative figures):**

| Item | Amount (₹) |
|---|---|
| Buy price of shares | 1,000 |
| Sell price of shares | 1,200 |
| Gross profit | 200 |
| Brokerage charge | 100 |
| **Customer's final profit** | **100** |

The BA's job here is to understand *what the business wants* and develop a platform for the client as per that requirement — not yet *how* it will be built.

> **Trainer tip:** Similar platforms include Upstox and Groww. Ask the class what business rules a trading app must capture (market hours, KYC, brokerage).

## Sample BRS document – Zerodha Kite

**A BRS states what the business needs, in business language**

**Business Requirement Specification (BRS)**

Project: Kite online trading platform · Prepared by: Business Analyst · For: Client (stock broker)

| | |
|---|---|
| **1. Purpose** | Let retail customers buy and sell shares online through web and mobile. |
| **2. Business objectives** | Grow the customer base and earn brokerage on every trade;<br>show customers their clear profit after charges. |
| **3. Scope** | In scope: account login, watchlist, buy/sell orders, holdings.<br>Out of scope: mutual funds, IPO applications (later release). |
| **4. Stakeholders** | Client (broker), end customers, BA, development, testing, support teams |
| **5. Business requirements** | BR-01 Customer can log in securely<br>BR-02 Customer can buy and sell listed shares<br>BR-03 Customer can view holdings and profit / loss |
| **6. Business rules** | Final profit = Sell value − Buy value − Brokerage<br>e.g. ₹1,200 − ₹1,000 − ₹100 = ₹100 final profit to the customer |
| **7. Assumptions** | Trading only during market hours; customer KYC already completed |
| **8. Sign-off** | Client and BA sign off before the SRS is written |

Sample BRS document · Zerodha Kite · 8 sections

Notice there are no screens, fields or validations here — only business needs, scope and rules. Those details arrive in the SRS.

# Phase 2 – Analysis (SRS)

The BA derives the **SRS** from the BRS. The SRS is also called the **FRS** (Functional Requirement Specification) or **CRS** (Customer Requirement Specification).

- The SRS is derived from the BRS and is prepared by the BA.
- It defines the **functional requirements** to be developed and the **system requirements**.
- It is shared with both the development team and the testing team.
- Output of the phase: the **SRS / FRS / CRS document**.

## What the SRS contains

| Section | What it means | Zerodha Kite example | Paytm example |
|---|---|---|---|
| Functional requirements | Attributes / fields that must be developed | User ID, password, PIN / OTP fields, Login button, Buy and Sell buttons | Login ID, password, Submit button |
| Functional flow diagram | The flow of tasks the customer performs | Login → watchlist → select stock → Buy → place order → confirmation | Login → recharge → number → operator → amount → payment → success |
| Use cases | One single requirement of the project, with description and acceptance criteria | Place a buy order | Mobile recharge module |
| Screenshot / prototype | A visual model of the screens before development | Mock-up of the order window | Mock-up of the recharge page |

# Sample SRS document – Zerodha Kite buy order

**The SRS turns each business requirement into testable functions**

**Software Requirement Specification (SRS / FRS)**

Module: Kite buy order · Derived from: BRS v1.0 · Prepared by: Business Analyst

| | |
|---|---|
| **1. Introduction** | Covers the buy-order module; shared with development and testing teams |
| **2. Functional reqs** | FR-01 Login with User ID, password and PIN / app code<br>FR-02 Search a stock and add it to the watchlist<br>FR-03 Place Buy order: quantity, Market / Limit, product type |
| **3. Functional flow** | Login → watchlist → select stock → Buy →<br>enter quantity and price → confirm → order placed message |
| **4. Use case** | UC-01 Place buy order: logged-in customer buys shares from watchlist |
| **5. Acceptance criteria** | Quantity must be a whole number greater than 0<br>Limit order must have a price; block the order if funds are insufficient |
| **6. System reqs** | Runs on web browser and mobile app<br>Built in .NET with a SQL Server database |
| **7. Prototype** | Mock-ups of login, watchlist and order window screens (made in Iris) |
| **8. Traceability** | BR-02 → FR-02, FR-03 → UC-01 → test cases TC_03 to TC_06 |

Sample SRS document · Kite buy-order module · 8 sections

Compare it with the BRS above: BR-02 ("customer can buy and sell shares") has become concrete fields, a flow, a use case and acceptance criteria that a tester can turn straight into test cases.

# Functional flow – Zerodha Kite buy order

1. Customer logs in with User ID and password.
2. Customer completes the second factor (PIN or app code).
3. Dashboard and watchlist open.
4. Customer searches for and selects a stock (e.g. INFY).
5. Customer clicks **Buy**; the order window opens.
6. Customer enters quantity, price type (Market / Limit) and product type.
7. Customer confirms the order.
8. The app shows an order-placed message; the stock appears in orders / positions.

## Functional flow – Paytm mobile recharge

1. Login with ID and password.
2. Open Mobile Recharge.
3. Enter mobile number (10 digits).
4. Select network (Idea, Airtel, etc.).
5. Select region (e.g. MH).
6. Enter recharge amount.
7. Choose payment method (UPI, net banking, debit card, credit card).
8. Recharge successful message is shown.

## Use case – parts of a use case

**A) Description** – the detailed requirement of that one use case.

- Kite: *Place Buy Order* – a logged-in customer can buy shares of a listed stock from the watchlist by entering quantity and price.
- Paytm: *Recharge module* – a customer can recharge a prepaid mobile number.

**B) Acceptance criteria** – the do's and don'ts for the requirement.

| Do's | Don'ts |
|---|---|
| Kite: quantity must be a whole number greater than 0 | Kite: order must not be placed if funds are insufficient |
| Kite: a Limit order must require a price | Kite: Buy button must not work when the user is logged out |
| Paytm: mobile number must be exactly 10 digits | Paytm: mobile number must not be less than 10 digits |

## Sample use case card from a real SRS

The card below is how one use case looks inside an SRS: a title, a value statement ("As a … I want … so that …"), acceptance criteria written as Given / When / Then, and a Definition of Done.

| | |
|---|---|
| **Title:** | Customer Inter Account Transfer |
| **Value Statement:** | As a bank customer,<br>I want to transfer funds between my linked accounts,<br>So that I can fund my credit card. |
| **Acceptance Criteria:** | *Acceptance Criterion 1:*<br>Given that the account is has sufficient funds<br>When the customer requests an inter account transfer<br>Then ensure the source account is debited<br>AND the target account is credited.<br><br>*Acceptance Criterion 2:*<br>Given that the account is overdrawn,<br>When the customer requests an inter account transfer<br>Then ensure the rejection message is displayed<br>And ensure the money is not transferred. |
| **Definition of Done:** | • Unit Tests Passed<br>• Acceptance Criteria Met<br>• Code Reviewed<br>• Functional Tests Passed<br>• Non-Functional Requirements Met<br>• Product Owner Accepts User Story |
| **Owner:** | MR I Owner |
| **Iteration:** | Unscheduled |
| **Estimate:** | 5 Points |

**How a tester reads this card:** each acceptance criterion becomes at least one test case. Criterion 1 is a positive test (sufficient funds → transfer succeeds); Criterion 2 is a negative test (overdrawn account → rejection message, no money moved).

### Prototype

The BA creates a prototype (screenshot / snapshot model) so the client can visualise the functionality before development begins. In this project the BA uses the **Iris** software for prototypes.

> **Tester's role in this phase:** once the SRS reaches the testing team, testers start **test case design**.

## Phase 3 – Design (HLD & LLD)

The design is prepared by the **designer / architect**, who produces two levels of design from the SRS.

| | HLD – High Level Design | LLD – Low Level Design |
|---|---|---|
| Focus | The main modules / screens and how they connect | The detailed logic inside each module |
| Level | Overall system view | Field, validation and component level |
| Used by | Architects, team leads | Developers (coding is done from the LLD) |

### HLD – Zerodha Kite buy-order module

1. Login screen – User ID and password text boxes
2. Two-factor screen – PIN / app code
3. Watchlist / dashboard – stock search and list
4. Order window – Buy / Sell, quantity, price, product type
5. Order confirmation – order placed message and order book

### HLD – Paytm recharge module

1. Mobile number text box and password text box
2. Circle (region) selection
3. Plan selection
4. Payment page
5. Success message / thank-you message

### LLD – Zerodha Kite order window (sample)

- **Quantity field:** numeric only; minimum 1; error message if blank or 0.
- **Price field:** enabled only for Limit orders; must be greater than 0.
- **Buy button:** enabled only when all mandatory fields are valid; calls the order-placement service.
- **Funds check:** if available margin is less than order value, show an "insufficient funds" message and block the order.

## Phase 4 – Coding

In the coding stage, **developers work on the LLD** and build the end-to-end functionality.

### Paytm recharge – what the developer builds from the LLD

Login ID → Password (strong, e.g. Abcd@123) → Submit button → Paytm home page opens → Recharge icon is shown → Recharge module opens as a new pop-up → recharge activities work end to end.

### Zerodha Kite – what the developer builds from the LLD

Login ID → Password → PIN / app code → Dashboard and watchlist → stock search → Buy / Sell order window with all validations → order placed and shown in the order book.

In this project the developers use **.NET**, and the database is **SQL Server**.

When development is complete, the build is deployed to the **SIT (System Integration Testing) environment** and the testing team is informed.

## Phase 5 – Testing

Testing is performed by the **testing team**. Actual testing starts after development is complete and the build is deployed to the **SIT environment**. As testers, the two main activities are **test case design** and **test case execution**.

| Activity | When it happens | Based on |
|---|---|---|
| Test case design | As soon as the SRS is received (during design and coding) | SRS – use cases and acceptance criteria |
| Test case execution | After the build is deployed to SIT | The test cases written earlier |

### Sample hand-over email from the development team

> Hi ABC (Tester),
>
> We have completed development end to end. You can start testing in the SIT environment.
>
> URL: 111.123/paytm/recharge

### Sample test cases – Zerodha Kite buy order

| TC ID | Scenario | Test data | Expected result |
|---|---|---|---|
| TC_01 | Login with valid User ID and password | Valid credentials | Second-factor screen is shown |
| TC_02 | Login with wrong password | Invalid password | Error message; user is not logged in |
| TC_03 | Place a Market buy order with sufficient funds | Qty = 10 | Order placed message; order visible in order book |
| TC_04 | Place an order with quantity 0 | Qty = 0 | Validation error; order not placed |
| TC_05 | Place an order with insufficient funds | Order value > available margin | "Insufficient funds" message; order blocked |
| TC_06 | Limit order without price | Price blank | Validation error on price field |

### Tools used in this project

- **Automation testing:** Java + Selenium WebDriver
- **Database testing:** SQL Server

## Phase 6 – Maintenance / Support

After the software goes live, the **support team** works on issues raised by real users in the live (production) application. Each issue is raised as a **ticket**, and the support team works on it until it is resolved.

| Application | Issue reported by the customer | What support does |
|---|---|---|
| Paytm | Recharge failed, but the transaction shows successful and money is deducted | Works on the ticket: checks the transaction, coordinates the refund or recharge, finds the root cause |
| Zerodha Kite | Buy order shows executed, but shares are not visible in holdings | Works on the ticket: checks order and holdings data, fixes the sync issue, informs the customer |
| Zerodha Kite | Brokerage charged is higher than expected in the contract note | Verifies the charge calculation against the business rule in the BRS |

Maintenance also includes small enhancements and fixes that keep the existing application running smoothly.

## Interview questions and model answers

**Q1. What is your team size?** I work in the project team, which has 23 people. Many people are involved in the project; I am part of the testing team.

**Q2. What is SDLC?** SDLC (Software Development Life Cycle) is the step-by-step process used to build software. It has six phases: Information gathering (BRS), Analysis (SRS), Design (HLD/LLD), Coding, Testing, and Maintenance/Support. Each phase produces an output that becomes the input for the next.

**Q3. What do you mean by the SRS / FRS / CRS document?** SRS stands for Software Requirement Specification, also called Functional Requirement Specification or Customer Requirement Specification. The BA derives it from the BRS. It defines the functional and system requirements and contains functional requirements, functional flow diagrams, use cases (description and acceptance criteria) and prototypes.

**Q4. When do you start testing?** When the BA prepares the SRS and sends it to the development and testing teams, we start test case design. When coding is complete and the software is deployed to the SIT environment, we start actual testing (test case execution).

**Q5. What is BRS? Have you handled the BRS?** BRS is the Business Requirement Specification, prepared by the BA after collecting requirements from the client. It contains the total business requirement. No, I have not handled the BRS; the BA owns it, and as a tester I work from the SRS.

**Q6. Which technologies are used in your project?** Development is done in .NET. For testing, automation uses Java with Selenium WebDriver, and database testing is done on SQL Server.

**Q7. What is the difference between HLD and LLD?** HLD shows the main modules and how they connect — for example, the Kite login, watchlist, order window and confirmation screens. LLD gives the detailed logic inside each module, such as field validations, and developers code from the LLD.

# Quick revision

| # | Phase | Done by | Output | Zerodha Kite example |
|---|---|---|---|---|
| 1 | Information gathering | BA (with client) | BRS | Buy/sell shares online; brokerage rule; profit after charges |
| 2 | Analysis | BA | SRS / FRS / CRS | Login fields, buy-order flow, use cases, acceptance criteria, prototype |
| 3 | Design | Designer / architect | HLD & LLD | Screens: login → watchlist → order window; field validations |
| 4 | Coding | Developers | Working build (deployed to SIT) | Order window built end to end in .NET |
| 5 | Testing | Testers | Test cases, execution results, defects | TC_01–TC_06 on login and buy order |
| 6 | Maintenance / support | Support team | Resolved tickets, fixes | Shares bought but not visible in holdings |